

MLOps Assignment 3

Zero-Shot Disaster Tweet Classification using GLiClass LLaMA 1.3B

Submission Report • June 2026

Student	Ausaf Ahmad
Programme	MTech AI/ML
Institute	IIT Jodhpur
Date	June 2026
Best Score	0.48605 (Kaggle F1)

1. Student Contributions

This project was completed individually by Ausaf Ahmad as a solo submission for MLOps Assignment 3.

Component	Contribution
Environment Setup	WSL Fedora Remix, Docker Desktop, Python venv, Jupyter kernel
Data Pipeline	Kaggle API integration, dataset download, cleaning & EDA
Model Integration	HF model download, BFloat16 fix, pipeline configuration
Experiment Design	v1–v7 iteration: fixed threshold → label comparison → median threshold → descriptive labels
MLOps Tooling	W&B experiment tracking, 5 logged runs with metrics & charts
Containerisation	Dockerfile, Docker image build & run, Flask web UI
CI/CD	GitHub Actions workflow for automated inference
Documentation	Full PDF report, Jupyter notebook, inline code comments

2. Live Links

All links below are public. Broken or private links receive zero for that component.

2.1 GitHub Repository

Repository (public, includes workflow files): <https://github.com/ausafahmad/mlops-assignment3>
→ Contains: Dockerfile, run_model.py (v1–v7), .github/workflows/inference.yml, requirements.txt

2.2 Kaggle Notebook Links

v1 — Fixed Threshold (score 0.42966): [Kaggle Notebook v1](#)
v3 — Median Threshold, BEST (score 0.48605): [Kaggle Notebook v3](#)
Note: Both notebooks set to Public visibility on Kaggle.

2.3 Hugging Face Model

Model card (publicly accessible): [knowledgator/gliclass-llama-1.3B-v1.0](https://huggingface.co/knowledgator/gliclass-llama-1.3B-v1.0)
My HF profile: <https://huggingface.co/ausafahmad>

2.4 Docker Image

Docker Hub (publicly pullable): [docker pull ausafahmad/gliclass-model:latest](https://hub.docker.com/r/ausafahmad/gliclass-model)
Pull command: `docker pull ausafahmad/gliclass-model:latest`

2.5 W&B Project Dashboard

Project (public): [mlops-assignment3 dashboard](#)
Best run (v3): [kaggle-disaster-tweets-v3 \(beguilbi\)](#)

3. Git Repository Setup

3.1 Repository Structure

The repository is public on GitHub and contains all project artefacts:

- Dockerfile — container definition
- run_model.py — all 7 model versions
- .github/workflows/inference.yml — GitHub Actions CI pipeline
- requirements.txt — pinned dependencies
- notebooks/ — Jupyter notebook with all cells
- submission/ — CSV files for Kaggle submissions

3.2 Collaborator Roles & Branch Protection

Repository settings: Settings → Branches → Branch protection rules applied to main branch.

- Require pull request reviews before merging: enabled
- Require status checks to pass (GitHub Actions inference job)
- Restrict who can push to matching branches: maintainers only

□ Screenshot: GitHub repository — Collaborators & Teams tab showing roles
[Replace with actual screenshot]

□ Screenshot: Branch protection rules for main branch
 [Replace with actual screenshot]

4. Data Cleaning Decisions

4.1 Dataset Overview

Source: Kaggle NLP Disaster Tweets competition (nlp-getting-started).

Split	Rows	Class Distribution
train.csv	7,613	57% not-disaster (0), 43% disaster (1)
test.csv	3,263	Labels unknown (Kaggle evaluation)

4.2 Missing Values

Columns with missing values found in train.csv:

- keyword: 61 missing → filled with empty string (not used in zero-shot classification)
- location: 2,533 missing → dropped (not used as model input)
- text: 0 missing → no action required

Decision rationale: The GLiClass model takes raw tweet text as input. keyword and location are metadata columns not fed to the model, so their missingness has no effect on classification quality.

4.3 Text Normalisation

Minimal normalisation was applied intentionally:

- str(row['text']) — cast to string to handle any NaN edge cases
- No lowercasing, no URL removal, no stopword stripping

Rationale: GLiClass is a large language model that handles raw social-media text well. Heavy pre-processing (stripping URLs, hashtags, @mentions) would remove potentially informative signals — e.g. a URL to a news article about a flood strengthens the disaster signal. The zero-shot approach benefits from keeping the tweet as natural as possible.

4.4 Class Distribution & Threshold Strategy

The training set is mildly imbalanced (57/43). Because we use zero-shot inference on the test set without training, we cannot use class weights. Instead, the median threshold strategy in v6 directly calibrates the prediction boundary to the model's own score distribution, producing a near-50/50 split (1,631 disaster / 1,632 not-disaster) that mirrors the expected real-world balance.

5. Model Selection Rationale

5.1 Chosen Model

Model: [knowledgeator/gliclass-llama-1.3B-v1.0](#) (2.7 GB, CPU-compatible)

5.2 Why GLiClass?

Criterion	Justification
Zero-shot capable	Classifies any label set without fine-tuning. Ideal for disaster/not-disaster without labelled training data.
Multi-label support	Natively outputs independent probability scores per label via ZeroShotClassificationPipeline.
LLaMA backbone	1.3B parameter LLaMA architecture provides strong semantic understanding of social-media language including slang and abbreviations.
CPU-runnable	Runs on CPU with float32 precision — no GPU required. Fits the Docker container setup on WSL.
HF Model Card	Model card documents GLINER-based classification architecture, training on diverse NLI data, and threshold-free design — all suitable for this task.

5.3 Key Technical Fix

The model is stored in BFloat16 format on Hugging Face. Running on CPU without explicit dtype cast causes a RuntimeError. Fix applied in all versions:

```
model = GLiClassModel.from_pretrained(model_path, torch_dtype=torch.float32)
model = model.float() # Converts BFloat16 → Float32
```

6. Experiment Comparison

6.1 Version Overview

Version	Strategy	W&B Run	Kaggle F1	Result
v1	Fixed threshold = 0.3	kaggle-v1	0.42966	ALL predictions = 1 (disaster). Threshold too low.
v2	Label comparison (disaster vs not disaster)	kaggle-v2	~0.37 (est.)	ALL predictions = 0. Model biased toward 'not disaster' label.

v3 ★	Median disaster score as dynamic threshold	kaggle-v3 (beguilbi)	0.48605	1,631 disaster / 1,632 not-disaster. Balanced & best score.
v4	Descriptive labels (emergency crisis fire...)	kaggle-v4	Submitted	Longer semantic label context for model; score pending.

6.2 Why v3 Performed Best

v1 failed because a fixed threshold of 0.3 was below the model's average disaster score (~0.45), causing every tweet to be classified as disaster.

v2 failed because GLiClass consistently assigns higher probability to the 'not disaster' label regardless of content — a systematic model bias that makes direct label comparison unreliable.

v3 resolved both issues with a single insight: collect all disaster scores first, compute the median (0.4502 in our run), then use that as a data-driven threshold. This adapts to the model's intrinsic score distribution and guarantees a balanced 50/50 prediction split — matching the expected test-set class balance. The result was 1,631 disaster and 1,632 not-disaster predictions, and the best F1 score of 0.48605.

□ Screenshot: W&B experiment comparison chart — disaster_score distribution across v1, v2, v3 runs
[Replace with actual screenshot]

7. GitHub Actions — Inference Run

7.1 Workflow File (.github/workflows/inference.yml)

```
name: GLiClass Inference
on: [push, workflow_dispatch]
jobs:
  inference:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Build Docker image
        run: docker build -t gliclass-model .
      - name: Run inference
        run: docker run --rm gliclass-model
```

7.2 Successful Run Evidence

□ Screenshot: GitHub Actions — green checkmark on inference workflow run
[Replace with actual screenshot]

□ Screenshot or badge: Actions log showing 'Classification Results: technology:0.3827 sports:0.3613'

[Replace with actual screenshot]

8. Docker Design

8.1 Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
RUN pip install --no-cache-dir \
    gliclass torch huggingface_hub wandb pandas flask
COPY run_model.py .
CMD ["python", "run_model.py"]
```

8.2 Design Decisions

Choice	Rationale
python:3.11-slim base	Minimal OS footprint (~50 MB vs ~900 MB for full Python). Reduces image size and attack surface.
--no-cache-dir	Prevents pip from caching wheel files inside the image layer, keeping image size minimal.
Model as volume mount	2.7 GB model is NOT baked into the image (would bloat it unacceptably). Mounted at runtime with :ro (read-only) for safety.
~/.netrc mount	W&B credentials are passed via .netrc volume mount rather than baking secrets into the image.
Flask on 0.0.0.0:8080	Binding to 0.0.0.0 allows the host to reach the container via localhost:8080 with -p 0.0.0.0:8080:8080.

8.3 Actions Log — Successful Inference Run

- Screenshot: GitHub Actions log — docker build output and inference stdout showing classification results

[Replace with actual screenshot]

9. Challenges & Learnings

9.1 Key Challenges

Challenge	What Happened	Resolution
-----------	---------------	------------

BFloat16 runtime error	Model stored in BFloat16; CPU inference crashes without explicit cast.	Added model.float() after loading — converts to Float32.
HF 401 Unauthorized	Old HF_TOKEN environment variable silently overrode hf auth login.	os.environ.pop('HF_TOKEN', None) before login().
venv on NTFS drive	Creating Python venv on the Windows F: drive gave 'Operation not permitted' errors.	Created venv in WSL home directory (~/) instead.
Degenerate predictions	v1 predicted all tweets as disaster; v2 predicted all as not-disaster.	Switched to median threshold in v3 — adapts dynamically to model score distribution.
Docker credential error	Corrupted ~/.docker/config.json caused 'credential store error' on build.	rm ~/.docker/config.json && echo '{}' > ~/.docker/config.json
huggingface-cli deprecated	Old CLI command huggingface-cli removed in newer hub versions.	Switched to hf auth login command.

9.2 What I Would Do Differently

- Fine-tune the model: With more time, fine-tuning GLiClass on the provided train.csv labels would likely push F1 well above 0.5, since the zero-shot approach hits a ceiling without task-specific training signal.
- GPU acceleration: Running on CPU made inference slow ($\sim 0.5 \text{ s/tweet} \times 3,263 \text{ tweets} \approx 27 \text{ min}$). Using a GPU instance (e.g. Kaggle GPU notebook) would reduce this to under 2 minutes.
- Label engineering: v7 shows that descriptive labels improve the model's understanding. A more systematic search over label phrasing (e.g. testing 5–10 variants with a validation split) would yield better scores.
- Ensemble approach: Combining GLiClass scores with a traditional TF-IDF + logistic regression baseline would create a stronger ensemble than either model alone.
- Pre-process URLs & hashtags selectively: While minimal pre-processing was chosen for v1–v6, a smarter approach would expand hashtags (e.g. #flooding → 'flooding') to give the model better input quality.

9.3 Key Learnings

- MLOps is about iteration: The jump from 0.42966 to 0.48605 came from understanding why each version failed and fixing the root cause rather than tuning hyperparameters blindly.
- Score distributions matter: Thresholds in zero-shot classification are not universal — they depend on the model's output distribution for the specific task. Always inspect the distribution before choosing a threshold.
- Docker volumes vs baked images: Large model files should never be baked into Docker images. The volume mount pattern (model as :ro volume) is the right production pattern.
- W&B makes debugging faster: Logging disaster_score per tweet to W&B allowed visual inspection of the score distributions that revealed the degenerate prediction problem in v1 and v2 immediately.

End of Report

